

# Interventions and Scenarios

Dynamic control and parameter exploration in AlgebraicABMs.jl

Simon Frost

2026-03-30

## 1 Overview

Real epidemiological models need **interventions** (vaccination campaigns, quarantines) and **scenario analysis** (comparing transmission rates). AlgebraicABMs.jl provides:

1. **Scheduled interventions** — fire at a fixed time
2. **Conditional interventions** — fire when a predicate becomes true
3. **Segmented simulation** — pause, modify state, and resume
4. **Parameter scenarios** — compare outcomes under different parameters
5. **Tie-breaking policies** — control behavior when events fire simultaneously

## 2 Setup

```
using AlgebraicABMs
using AlgebraicABMs.ABMs: RuntimeABM, Traj
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

TaskLocalRNG()
```

### 2.1 Shared model components

We build a simple graph-based model: vertices are individuals, edges are contacts. Infection removes an edge.

```
# Infection rule: remove an edge (contact consumed)
L_inf = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end
I_inf = Graph(2)
R_inf = Graph(2)
inf_rule = Rule(
    homomorphism(I_inf, L_inf; initial=(V=[1,2],)),
    homomorphism(I_inf, R_inf; initial=(V=[1,2],)))

# Vertex duplication rule (for population growth examples)
v1 = Graph(1)
v2 = Graph(2)
dup_rule = Rule(id(v1), homomorphism(v1, v2; initial=(V=[1],)))

# Helper: create a complete graph
function complete_graph(n)
```

```

@acset Graph begin
    V = n
    E = n * (n - 1)
    src = vcat([[i for _ in 1:(n-1)] for i in 1:n]...)
    tgt = vcat([[j for j in 1:n if j != i] for i in 1:n]...)
end
end

init = complete_graph(8)
println("Initial state: $(nparts(init, :V)) vertices, $(nparts(init, :E)) edges")

Initial state: 8 vertices, 56 edges

```

### 3 1. Scheduled Interventions

A **scheduled intervention** fires at a fixed time, modifying the state. After the intervention, all clocks are automatically refreshed.

**Example:** A vaccination campaign at day 10 that removes half the contact edges (social distancing).

```

infection = ABMRule(:Infection, inf_rule, ContinuousHazard(0.1))
abm = ABM([infection])

# At t=10, remove half the edges
vaccination = Intervention(10.0, state -> begin
    ne = nparts(state, :E)
    n_remove = div(ne, 2)
    if n_remove > 0
        rem_parts!(state, :E, 1:n_remove)
    end
end; name=:vaccination)

traj = run!(abm, deepcopy(init); maxtime=25.0, interventions=[vaccination])

println("Events: $(length(traj))")
for (i, (t, _, name, _)) in enumerate(traj.events)
    i > 15 && break
    println(" t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
Events: 56
t=0.001: Infection
t=0.002: Infection
t=0.004: Infection
t=0.01: Infection
t=0.01: Infection
t=0.014: Infection
t=0.017: Infection
t=0.017: Infection
t=0.021: Infection
t=0.025: Infection
t=0.029: Infection
t=0.036: Infection
t=0.037: Infection

```

```
t=0.039: Infection
t=0.046: Infection
```

Note how the event rate changes around  $t=10$ : fewer edges means fewer potential infection events.

### 3.1 Multiple scheduled interventions

```
# Two waves of contact reduction
wave1 = Intervention(5.0, state -> begin
  ne = nparts(state, :E)
  n_remove = div(ne, 3)
  n_remove > 0 && rem_parts!(state, :E, 1:n_remove)
end; name=:wave1)

wave2 = Intervention(15.0, state -> begin
  ne = nparts(state, :E)
  n_remove = div(ne, 3)
  n_remove > 0 && rem_parts!(state, :E, 1:n_remove)
end; name=:wave2)

traj_multi = run!(abm, deepcopy(init); maxtime=30.0, interventions=[wave1, wave2])
println("Two-wave intervention: $(length(traj_multi)) events")

[ Info: Stochastic scheduling algorithm ran out of events
Two-wave intervention: 56 events
```

## 4 2. Conditional Interventions

A **conditional intervention** fires when a predicate on the state becomes true (checked after each event).

**Example:** Quarantine — when fewer than 20 edges remain, add 3 new vertices (incoming aid workers).

```
# Conditional: if edges drop below 20, add vertices
quarantine = Intervention(
  state -> nparts(state, :E) < 20,
  state -> add_parts!(state, :V, 3);
  name=:quarantine
)

traj_cond = run!(abm, deepcopy(init); maxtime=30.0, interventions=[quarantine])

println("Conditional intervention: $(length(traj_cond)) events")
if !isempty(traj_cond.hist)
  final = codom(right(last(traj_cond.hist)))
  println("Final: $(nparts(final, :V)) vertices, $(nparts(final, :E)) edges")
end

[ Info: Stochastic scheduling algorithm ran out of events
Conditional intervention: 56 events
Final: 68 vertices, 0 edges
```

### 4.1 Combining scheduled and conditional

```
# Vaccination at t=8 + emergency quarantine if edges drop too low
combined = [
  Intervention(8.0, state -> begin
```

```

        ne = nparts(state, :E)
        n_remove = div(ne, 2)
        n_remove > 0 && rem_parts!(state, :E, 1:n_remove)
    end; name=:vaccinate),
    Intervention(
        state -> nparts(state, :E) < 10,
        state -> add_parts!(state, :V, 2);
        name=:emergency
    )
]

traj_combo = run!(abm, deepcopy(init); maxtime=30.0, interventions=combined)
println("Combined interventions: $(length(traj_combo)) events")

[ Info: Stochastic scheduling algorithm ran out of events
Combined interventions: 56 events

```

## 5 3. Segmented Simulation with refresh\_clocks!

For fine-grained control, run the simulation in segments. Between segments, modify the state manually and call `refresh_clocks!` to rebuild the event queue.

```

# Phase 1: run for 5 time units
dup = ABMRule(:dup, dup_rule, ContinuousHazard(1.0))
abm_seg = ABM([dup])

rt = RuntimeABM(abm_seg, Graph(2))
traj_seg = run!(abm_seg, rt, Traj(Graph(2)); maxevent=3)

println("Phase 1: $(nparts(rt.state, :V)) vertices after $(length(traj_seg)) events")

# External modification: add 5 more vertices
add_parts!(rt.state, :V, 5)
println("After manual addition: $(nparts(rt.state, :V)) vertices")

# Rebuild clocks and continue
refresh_clocks!(rt, abm_seg)
traj_seg2 = run!(abm_seg, rt, traj_seg; maxevent=6)

println("Phase 2: $(nparts(rt.state, :V)) vertices after $(length(traj_seg2)) total events")

Phase 1: 5 vertices after 3 events
After manual addition: 10 vertices
Phase 2: 13 vertices after 6 total events

```

The `RuntimeABM` preserves simulation state (current time, event queue) across segments, while `refresh_clocks!` ensures the event queue reflects the modified state.

### 5.1 Multi-phase epidemic

```

# Phase 1: fast transmission (no control)
infection_fast = ABMRule(:InfFast, inf_rule, ContinuousHazard(0.05))
abm_p1 = ABM([infection_fast])

init_phases = complete_graph(8)

```

```

rt_phases = RuntimeABM(abm_p1, deepcopy(init_phases))
traj_phases = run!(abm_p1, rt_phases, Traj(deepcopy(init_phases))); maxtime=5.0

println("Phase 1 (no control): $(length(traj_phases)) events, $(nparts(rt_phases.state, :E)) edges remain")

# Phase 2: intervention removes edges, then continue with slower rate
ne = nparts(rt_phases.state, :E)
n_remove = div(ne, 2)
n_remove > 0 && rem_parts!(rt_phases.state, :E, 1:n_remove)

infection_slow = ABMRule(:InfSlow, inf_rule, ContinuousHazard(0.2))
abm_p2 = ABM([infection_slow])
rt_p2 = RuntimeABM(abm_p2, rt_phases.state)
rt_p2.tnow = rt_phases.tnow

traj_p2 = run!(abm_p2, rt_p2, traj_phases; maxtime=15.0)
println("Phase 2 (post-intervention): $(length(traj_p2)) total events, $(nparts(rt_p2.state, :E)) edges remain")

[ Info: Stochastic scheduling algorithm ran out of events
Phase 1 (no control): 56 events, 0 edges remain
[ Info: Stochastic scheduling algorithm ran out of events
Phase 2 (post-intervention): 56 total events, 0 edges remain

```

## 6 4. Parameter Scenarios with run\_scenarios

run\_scenarios runs the same model under multiple parameter sets, returning a vector of (scenario, trajectory) pairs.

```

# Parameterized infection rule
param_infection = ABMRule(:InfParam, inf_rule,
    ClosureParams((m, params) -> Exponential(params.beta)))

abm_scenarios = ABM([param_infection]; params=(beta=1.0,))

# Three scenarios: low, medium, high transmission
scenarios = [
    (beta=2.0,), # slow (high mean waiting time)
    (beta=0.5,), # medium
    (beta=0.1,), # fast (low mean waiting time)
]

results = run_scenarios(abm_scenarios, deepcopy(init), scenarios; maxtime=15.0)

println("Scenario comparison:")
for (scen, traj) in results
    n_events = length(traj)
    final_edges = if !isempty(traj.hist)
        nparts(codom(right(last(traj.hist))), :E)
    else
        nparts(init, :E)
    end
    println("    =$(scen.beta): $(n_events) events, $(final_edges) edges remaining")
end

[ Info: Stochastic scheduling algorithm ran out of events

```

```
[ Info: Stochastic scheduling algorithm ran out of events
[ Info: Stochastic scheduling algorithm ran out of events
Scenario comparison:
  =2.0: 56 events, 0 edges remaining
  =0.5: 56 events, 0 edges remaining
  =0.1: 56 events, 0 edges remaining
```

Higher (longer mean waiting time) leads to fewer events; lower leads to rapid transmission. This enables systematic exploration of the parameter space.

## 6.1 Comparing with observables

```
obs_v = Observable(:Vertices, Graph(1))
obs_e = Observable(:Edges, @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end)

println("\nDetailed scenario with observables:")
for (scen, _) in results
  abm_obs = ABM([param_infection]; params=scen)
  traj_obs = run!(abm_obs, deepcopy(init);
    maxtime=10.0,
    observables=[obs_v, obs_e],
    save_every=2.0)
  println("  =$(scen.beta):")
  for (t, counts) in traj_obs.snapshots
    println("    t=$(round(t, digits=1)): V=$(counts[:Vertices]), E=$(counts[:Edges])")
  end
end
```

```
Detailed scenario with observables:
[ Info: Stochastic scheduling algorithm ran out of events
  =2.0:
    t=0.0: V=8, E=56
    t=2.0: V=8, E=21
    t=4.0: V=8, E=6
    t=6.0: V=8, E=2
    t=8.0: V=8, E=0
[ Info: Stochastic scheduling algorithm ran out of events
  =0.5:
    t=0.0: V=8, E=56
    t=2.0: V=8, E=0
[ Info: Stochastic scheduling algorithm ran out of events
  =0.1:
    t=0.0: V=8, E=56
```

## 7 5. Tie-Breaking Policies

When multiple events fire at the exact same time (common with `DiscreteHazard`), the **tie-breaking policy** determines how they are handled.

### 7.1 TieBreak (default)

Execute events in arbitrary (deterministic) order. Later events may be invalidated if earlier ones remove their match.

```
# Two rules both fire at t=1
r_map = homomorphism(Graph(1), Graph(2); initial=(V=[1],))
dup1 = ABMRule(:dup1, Rule(id(Graph(1)), r_map), DiscreteHazard(1.0))
dup2 = ABMRule(:dup2, Rule(id(Graph(1)), r_map), DiscreteHazard(1.0))

abm_tb = ABM([dup1, dup2]) # default TieBreak
println("Tie policy: $(abm_tb.tiepolicy)")

traj_tb = run!(abm_tb, Graph(1); maxevent=1)
final_tb = codom(right(last(traj_tb.hist)))
println("After 1 event: $(nparts(final_tb, :V)) vertices")

Tie policy: TieBreak
After 1 event: 3 vertices
```

## 7.2 TieRandom

Shuffle the order of simultaneous events randomly. Useful when the arbitrary ordering might introduce systematic bias.

```
abm_tr = ABM([dup1, dup2]; tiepolicy=TieRandom)
println("Tie policy: $(abm_tr.tiepolicy)")

traj_tr = run!(abm_tr, Graph(1); maxevent=1)
final_tr = codom(right(last(traj_tr.hist)))
println("After 1 event: $(nparts(final_tr, :V)) vertices")

Tie policy: TieRandom
After 1 event: 3 vertices
```

## 7.3 TieError

Raise an error if simultaneous events occur. Useful during model development to verify that your timers avoid ties.

```
abm_te = ABM([dup1, dup2]; tiepolicy=TieError)
println("Tie policy: $(abm_te.tiepolicy)")

try
    run!(abm_te, Graph(1); maxevent=1)
    println("No error (unexpected)")
catch e
    println("Error caught: ", e.msg[1:min(60, end)])
end

Tie policy: TieError
Error caught: TieError policy: 2 simultaneous events at t=1.0
```

## 7.4 When ties don't occur

With a single rule or continuous timers, ties are (almost surely) impossible.

```
abm_single = ABM([dup1]; tiepolicy=TieError)
traj_single = run!(abm_single, Graph(1); maxevent=1)
final_single = codom(right(last(traj_single.hist)))
println("Single rule with TieError: $(nparts(final_single, :V)) vertices (no error)")

Single rule with TieError: 2 vertices (no error)
```

## 8 Summary

| Feature                         | API                                              | When to Use                                          |
|---------------------------------|--------------------------------------------------|------------------------------------------------------|
| <b>Scheduled intervention</b>   | <code>Intervention(time, action)</code>          | Vaccination campaigns, policy changes at known dates |
| <b>Conditional intervention</b> | <code>Intervention(predicate, action)</code>     | Quarantine triggers, capacity-based responses        |
| <b>Segmented run</b>            | <code>RuntimeABM + refresh_clocks!</code>        | Multi-phase simulations, manual state changes        |
| <b>Parameter scenarios</b>      | <code>run_scenarios(abm, init, scenarios)</code> | Sensitivity analysis, R estimation                   |
| <b>Tie-breaking</b>             | <code>ABM(rules; tiepolicy=...)</code>           | Control simultaneous event handling                  |

These tools support the full modeling workflow: build a base model, add realistic interventions, and explore the parameter space to inform public health decisions.